Who Wrote It? Identifying LLMs from Their Responses

CMPSC 448, Project 1 · 11 October 2026

Team leader: [name] · Members: [names, or "individual project"]

Repository: https://github.com/GraphoLogiCode/cmpsc448_project_1

Summary. Three open-weight LLMs (Llama 3.1, Qwen 2.5 and Mistral 7B) each answered the same 2,000 prompts, giving a balanced dataset of 6,000 answers. A CNN, an LSTM and a TF-IDF baseline were trained to name the model behind an answer. All three reach about 88% accuracy on held-out test prompts, against 33.3% for guessing, and they cannot be told apart from one another. The prompt alone gives exactly chance. When a whole task type is held out of training, accuracy falls to 65–70% on average, and to around chance on math for the two neural models, so much of what the classifiers learn is tied to the kind of question. The models can be recognised from their formatting alone and from their words alone; no single signal is necessary.

1. Problem definition

Given a piece of text written by a large language model, the task is to say which model wrote it. It is a three-class text classification problem: the input is text, the label is one of `llama3.1:8b`, `mistral:7b` and `qwen2.5:7b`. The classes are balanced, so a classifier that guesses is right one time in three.

The project answers four questions.

	Question
RQ1	Can the model be identified from its answer?
RQ2	Does the user's prompt help: prompt only, answer only, or both?
RQ3	Does a classifier trained on some kinds of task still work on another kind?
RQ4	What characteristics distinguish the models?

Two models are required, a convolutional network (CNN) and a recurrent network (LSTM). A TF-IDF model with logistic regression is added as a baseline, so that the neural models have a number to beat.

2. Dataset

2.1 Design

Every model answered the **same prompts under the same settings**. This choice matters more than any other in the project:

- The classes are balanced by construction: one answer per model per prompt.

- A difference between two answers to one prompt can only come from the model, not from the question.
- It makes one leak measurable. A classifier that sees only the prompt must score exactly one third, because each prompt appears three times with three different labels. A higher score would mean the data or the split is flawed.

2.2 Prompts

The 2,000 prompts cover four task types, 500 each, so that RQ3 can hold one type out.

Task	Source	Licence	Median length
Coding	C++ functions from four open-source game engines (Godot, OGRE, Torque3D, Esoterica; 125 each), each preceded by one fixed instruction to review the function	MIT	612 characters
Math	GSM8K word problems, test split	MIT	225
Factual	Dolly-15k, category `open_qa`	CC BY-SA 3.0	41
Writing	Dolly-15k, category `creative_writing`	CC BY-SA 3.0	69

The functions come from a separate research project on code smells in game engines. Only engines with a plain MIT licence were used, and only files whose header names the engine's own copyright holder. Prompts were sampled with a fixed seed, and exact repeats were removed. No prompt contains an email address or a phone number. Sources, selection rules, licences and attribution are recorded in `data/prompts/SOURCES.md`.

2.3 Models and generation

Model	Maker	Size	Build
`llama3.1:8b`	Meta	8.0B parameters	4-bit (Q4_K_M)
`qwen2.5:7b`	Alibaba	7.6B	4-bit (Q4_K_M)
`mistral:7b` (v0.3)	Mistral AI	7.2B	4-bit (Q4_K_M)

The three come from different makers and are close in size, so the classifier separates families and not a small model from a large one. They were run locally with Ollama 0.32.4 on one RTX 5090.

Setting	Value
System prompt	"You are a helpful assistant." for every model
Temperature, seed	0.7, seed 448
Longest answer	600 tokens
Answers per prompt	One

Two settings were decided by looking at the data first.

- **The system prompt is set explicitly.** Without one, Ollama uses the prompt each model was packaged with. Qwen's package says "You are Qwen, created by Alibaba Cloud"; the other two have none. Leaving the default would have told one model its own name in every request.
- **The token limit is 600.** In a trial, a limit of 300 cut off 50–75% of the answers. At 600 it was 10% for Llama and Mistral and 35% for Qwen.

Generating the 6,000 answers took 65 minutes. No request failed and no answer is empty.

2.4 Checks and cleaning

The raw answers were inspected before any model was trained. The script `src/check_responses.py` produces the counts.

Finding	Rows	What was done
Every Mistral answer begins with a space character	2,000	Whitespace is stripped from both ends of every answer. Left in, the first character alone would identify Mistral.
A Qwen answer contains its maker's name although the prompt does not: once as "As an AI developed by Alibaba Cloud...", once as the name of an online shop	2	The answer is removed, together with the other two answers to the same prompt, so the classes stay balanced.
Answers cut off at the 600-token limit	796	Kept. 598 of them are Qwen's, including 91% of its code reviews.
Answers that open with a refusal	9	Kept; this is real behaviour of the models.
Two models gave the identical sentence ("The capital of Arizona is Phoenix.")	2 prompts	Kept.

2.5 Split

The data is split **by prompt**, 70% / 15% / 15% within each task, so the three answers to a prompt always fall in the same split and no test prompt was seen in training.

Split	Prompts	Rows	Rows per model
Train	1,398	4,194	1,398
Validation	300	900	300
Test	300	900	300

3. Models and training

3.1 Input

Text is lower-cased and split into tokens: a token is a word, or one punctuation or markdown character. The vocabulary is built from the training split only (19,928 tokens for the answers), and a token seen fewer than twice becomes an unknown-word token. A text is cut to its first 512 tokens, which shortens 15.7% of the training answers. Batches hold 32 texts, padded to the longest one.

3.2 CNN

Layer	Detail	Parameters
Embedding	128 dimensions, learned from scratch	2,550,784
Convolutions	Four in parallel, widths 2, 3, 4 and 5, 128 filters each, ReLU	229,888
Pooling	Maximum over time of each filter, giving 512 numbers	0
Output	Dropout 0.5, then a linear layer to 3 scores	1,539
Total		2,782,211

A filter of width *k* responds to one kind of *k*-token pattern. The maximum over time asks whether the pattern occurs anywhere in the text, so the result does not depend on position or length. This follows Kim (2014).

3.3 LSTM

Layer	Detail	Parameters
Embedding	128 dimensions, then dropout 0.5	2,550,784
LSTM	One bidirectional layer, 256 units per direction	790,528
Pooling	Maximum over time of each of the 512 outputs	0
Output	Dropout 0.5, then a linear layer to 3 scores	1,539
Total		3,342,851

The LSTM (Hochreiter and Schmidhuber, 1997) reads the tokens in order and carries a memory from one to the next. Reading in both directions gives each position the context on both sides. Sequences are packed, so the LSTM stops at each text's real end. In both models padding is excluded from the pooling, so a prediction does not depend on the other texts in the batch.

3.4 Baseline

TF-IDF over single tokens and pairs of tokens, with the same tokens as the neural models, followed by logistic regression. Its regularisation strength is chosen on the validation split.

3.5 Training

Setting	Value
Loss and optimiser	Cross-entropy; Adam, learning rate 0.001
Batch size	32
Gradient clipping	Norm limited to 5
Model selection	The epoch with the best validation accuracy is kept
Early stopping	After 3 epochs without improvement, at most 20 epochs
Test split	Read once per run, after training

The CNN trains in about 10 seconds and the LSTM in about 90. The CNN's validation accuracy peaks at epoch 6; after that its training loss keeps falling while its validation loss rises, which is overfitting, and early stopping keeps epoch 6. The LSTM improves more slowly and peaks at epoch 11.

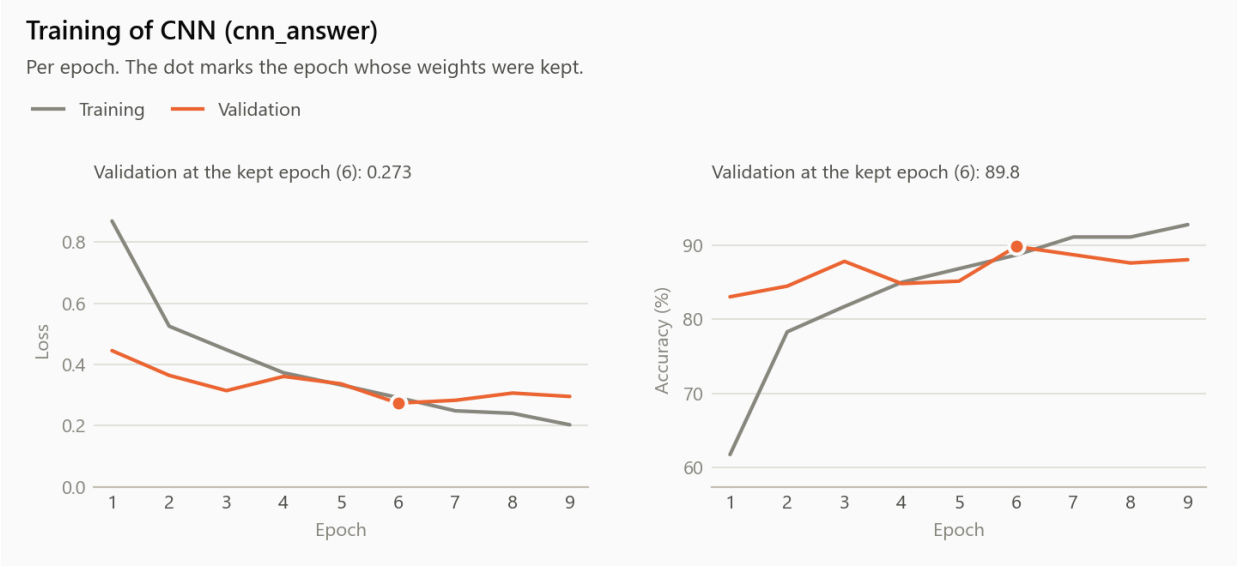


Figure 1. Loss and accuracy of the CNN per epoch. The dot marks the epoch whose weights were kept.

Two checks were made before trusting the code. Each model can memorise 50 training rows, which rules out a basic bug. And a model trained on the prompt alone stays at exactly one third.

3.6 Choice of settings

Twenty-four trials compared settings on the **validation** split only (`src/tune.py`).

- **CNN.** Eight variations (more tokens, other filter widths, more filters, larger embedding, less dropout, lower learning rate, larger batch) changed validation accuracy by at most one point. The starting settings were kept.
- **LSTM.** 256 units scored 88.7% against 86.9% for 128, and maximum over time scored 86.9% against 85.7% for using the final memory. Both were adopted.

The chosen settings were repeated with three other random seeds. Validation accuracy ranged from 88.0% to 89.9% for the CNN and from 86.8% to 88.7% for the LSTM, a spread of 1.9 points each. The differences between settings are therefore about as large as the difference between two runs of one setting.

4. Results

4.1 RQ1: can the model be identified from its answer?

Yes. Accuracy on the 900 test answers:

Classifier	Prompt only	Answer only	Prompt + answer
TF-IDF baseline	33.3%	88.3%	87.8%
CNN	33.3%	88.8%	87.6%
LSTM	33.3%	87.6%	87.1%

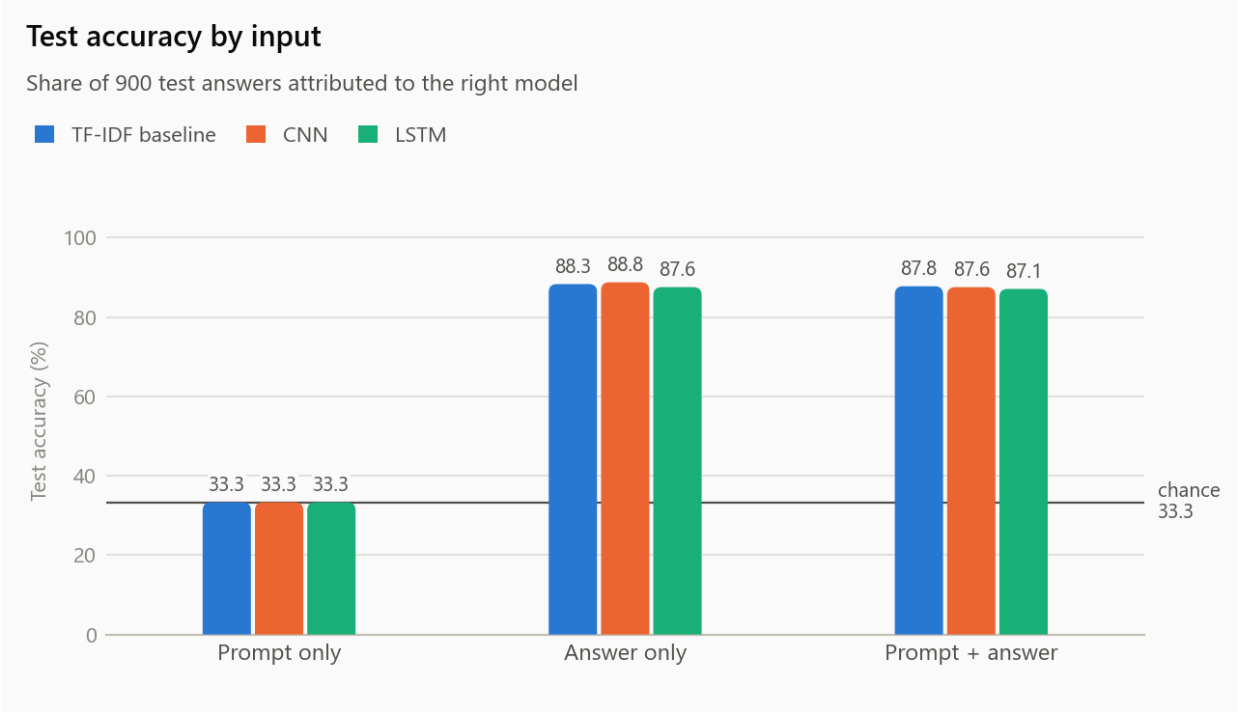


Figure 2. Test accuracy of the three classifiers for the three inputs. The line is the chance level.

The three classifiers are within about one point of each other, less than the spread between seeds, so none can be called better. The CNN and the LSTM also agree with each other: they give the same prediction for 815 of the 900 test answers, and 70 answers are wrong for both.

By model. Precision and recall on the test split, answer only:

Model	Baseline	CNN	LSTM
Llama	0.90 / 0.86	0.89 / 0.89	0.87 / 0.89
Mistral	0.80 / 0.95	0.83 / 0.93	0.85 / 0.88
Qwen	0.98 / 0.83	0.96 / 0.85	0.91 / 0.86

Mistral is the class the others are mistaken for: its recall is high and its precision low. Qwen is the reverse: a prediction of Qwen is almost always right, but about 15% of its answers go to another model.

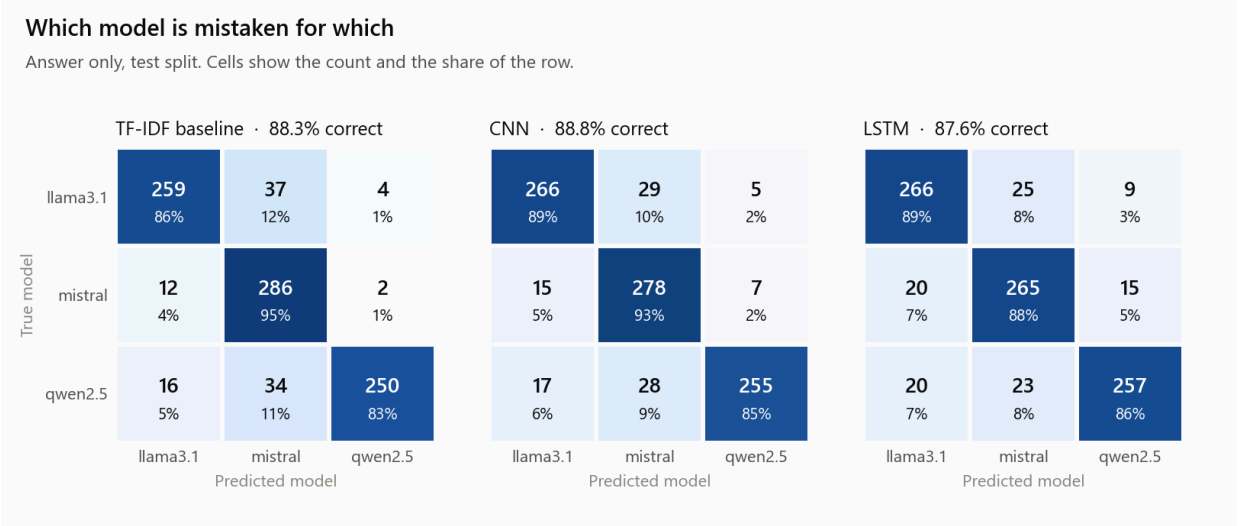


Figure 3. Confusion matrices on the test split, answer only.

By task. Code reviews are identified perfectly and short factual answers are hardest, for every classifier.

Task	Baseline	CNN	LSTM
Coding	100%	100%	100%
Math	93.3%	96.0%	93.3%
Writing	83.6%	82.2%	82.7%
Factual	76.4%	76.9%	74.2%

4.2 RQ2: does the prompt help?

No.

- **Can the model be predicted from the prompt alone?** No: all three classifiers score exactly 33.3%. Each prompt appears three times with identical text and three different labels, so any prediction is right for one row and wrong for two.
- **Does adding the prompt improve classification?** No. Accuracy with prompt and answer is 0.5 to 1.2 points below answer only for every classifier. The prompt carries no information about the model and uses part of the 512 tokens.
- **When would prompt-only classification work surprisingly well?** When the dataset lets the prompt reveal the label: if different models were given different prompts, as in logs of real conversations where each user chooses a model, or if a random split put the same prompt in training and test. A score above chance would then measure the dataset, not the models. Here that number is a built-in check, and it came out at exactly chance.

5. In-depth analyses

5.1 RQ3: do the fingerprints carry over to a new kind of task?

Each classifier was trained on three task types and tested on the fourth. "Seen" is the accuracy on that task when it was part of training (section 4.1).

Held-out task	Baseline: seen → held out	CNN	LSTM
Coding	100 → 76.4	100 → 94.2	100 → 96.9
Factual	76.4 → 67.1	76.9 → 64.4	74.2 → 67.1
Math	93.3 → 63.1	96.0 → 30.2	93.3 → 39.1
Writing	83.6 → 72.9	82.2 → 70.7	82.7 → 56.4
Mean	88.3 → 69.9	88.8 → 64.9	87.6 → 64.9

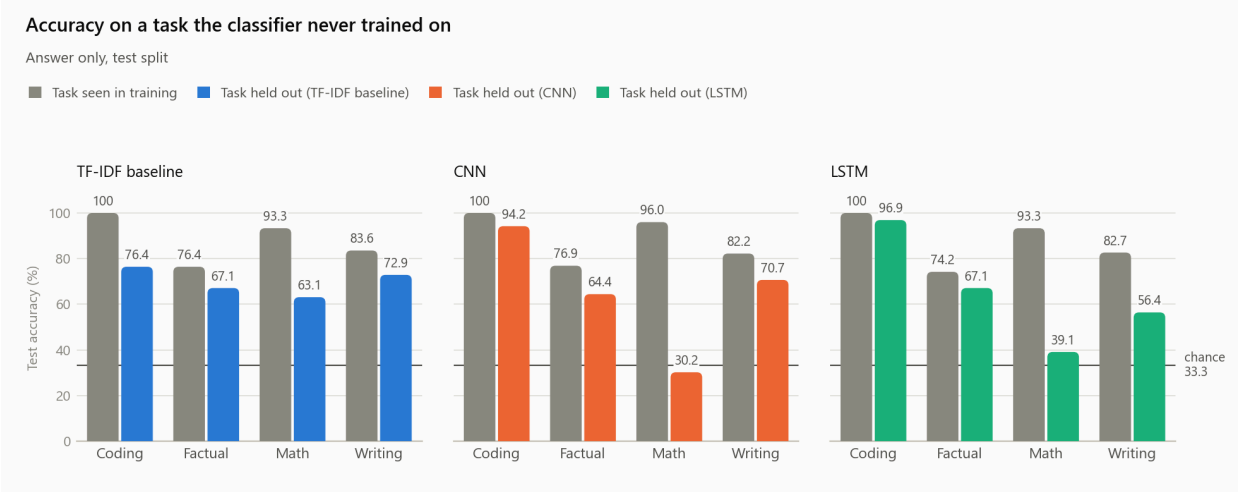


Figure 4. Accuracy on each task when it was seen in training (grey) and when it was held out (colour).

- **Does performance fall on unseen tasks?** Yes, in all twelve cases, by 3 to 66 points. On average accuracy drops from about 88% to 65–70%.
- **Is a model consistently identifiable across tasks?** No. Averaged over the four held-out tasks, recall is 0.58 to 0.72 for Llama, 0.68 to 0.77 for Mistral and 0.54 to 0.75 for Qwen, depending on the classifier, and the ranking changes with the classifier. Qwen shows the extremes: every classifier finds all of its held-out code reviews, and fewer than half of its held-out factual answers.
- **A general fingerprint, or task-specific patterns?** Mostly task-specific. Something carries over, since ten of the twelve cases are well above chance. But math shows how much does not: the CNN scores 30.2%, below chance. It labels 61 of Mistral's 75 math answers as Llama and 42 of Llama's 75 as Mistral, and finds only 23 of Qwen's 75.

A likely reason for the math result is that the models format math differently from everything else. Llama uses bold text in 52% of its answers overall but in 3% of its math answers, so a classifier that learned "plain text means Mistral" is misled. Qwen writes LaTeX in every math answer and almost

nowhere else, so those tokens are unknown to a classifier that never saw math. Why Mistral's math answers are taken for Llama's was not investigated.

One caveat: the held-out classifiers also train on a quarter less data (about 3,145 rows instead of 4,194), which may account for part of the drop.

5.2 RQ4: what distinguishes the models?

Style of each model (training split, 1,398 answers each):

	Llama	Mistral	Qwen
Mean length, tokens	281	265	373
Distinct words used	19,093	18,862	20,359
Distinct tokens among the first 100	60.6	61.7	60.3
Mean sentence length, words	20.8	22.6	17.2
Answers with bold text	51.6%	2.8%	64.0%
Answers with headings	0.8%	0.1%	35.0%
Answers with bullet lists	25.0%	9.7%	52.5%
Answers with LaTeX	0.9%	0.0%	25.8%
Answers with curly quotes	0.0%	0.1%	22.9%
Answers cut off at 600 tokens	4.2%	5.5%	30.2%

Formatting separates the models clearly: Mistral writes almost plain text, Llama uses bold, Qwen uses bold, headings, lists, LaTeX and typographic quotes. Lexical diversity does not: among the first 100 tokens all three use about 60 distinct ones. Qwen writes about a third more than the others.

Structural features alone. A logistic regression that sees only 16 numbers per answer (length, punctuation rates, counts of markdown and LaTeX markers) and no words reaches **78.2%**. Its strongest features are LaTeX markers, typographic quotes and headings for Qwen, and the absence of bold for Mistral. Length alone reaches 37.9%, barely above chance.

Removing one signal at a time. Each classifier was trained again on changed answers. Test accuracy:

Change to the answers	Baseline	CNN	LSTM
None	88.3%	88.9%	87.6%
Markdown and LaTeX markup removed	87.1%	84.1%	78.7%
All punctuation removed (words only)	83.3%	80.9%	76.7%
Every word replaced by a placeholder (structure only)	83.3%	83.9%	87.1%
Cut to the first 100 tokens	83.6%	82.8%	80.8%
First 30 tokens removed	86.9%	85.4%	85.0%
Only the first 30 tokens	77.7%	75.3%	75.8%

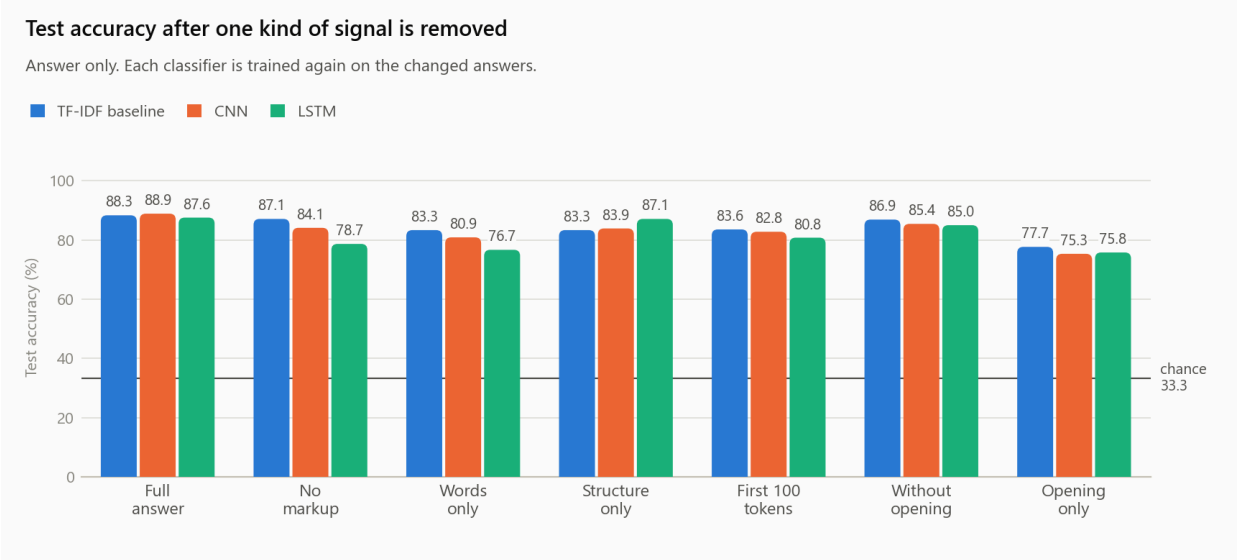


Figure 5. Test accuracy after one kind of signal is removed from the answers.

- **What signals do the classifiers use?** Markup, punctuation habits, characteristic openings and word choice. The baseline's largest weights are bold and list markers for Llama, phrases such as "let's find", "we first" and "first need" for Mistral, and `#`, the backslash, the typographic apostrophe and the long dash for Qwen. The most common openings differ too: "Let's break" for Llama, "To determine" for Qwen, "This C++" for Mistral.
- **Linguistic, structural, or both?** Both, and either is nearly enough. With every word replaced by a placeholder, so that only punctuation and layout remain, accuracy is still 83–87%. With all punctuation removed, so that only words remain, it is 77–83%.
- **Does removing a signal reduce accuracy?** Yes, but never by much. The largest loss from removing one kind of signal is 11 points (LSTM, words only), and no change brings any classifier below 75%. The signals are redundant: when one is taken away the classifiers find the model through another. Even the first 30 tokens alone give about 76%.
- **The LSTM leans on structure more than the others.** It loses 9 points without markup and almost nothing when the words are replaced.

5.3 How far the numbers can be trusted

- One point of test accuracy is 9 answers. A rough 95% interval around 88% is ±2 points, and the answers are not independent, because each prompt contributes three.
- The same model with a different random seed moves by up to 1.9 points on validation.
- Differences of one or two points between classifiers, or between settings, should therefore not be read as real. The large effects in RQ3 and RQ4 are far outside this range.

6. Lessons and experience

[Draft points taken from what happened in this project. Rewrite this section in your own words and add what you personally found hard or surprising.]

- **Look at the raw data before training anything.** Every Mistral answer began with a space, which a classifier could have used to reach a high score for a reason that has nothing to do with writing style. One model's packaged system prompt named the model. Neither would have been found by looking at accuracy alone.
- **Design the dataset so that a leak shows up as a number.** Giving every model the same prompts made "prompt-only accuracy" a test with a known correct answer, one third. That test later exposed a real bug: a prompt-only run that trained for all 20 epochs revealed that tied epochs were compared wrongly in the early-stopping code.
- **Start with a simple baseline.** TF-IDF with logistic regression matched both neural networks. With about 4,000 training texts and a signal that sits in which words and symbols occur, a larger model had nothing extra to find.
- **Measure the noise before comparing.** Repeating a run with another seed moved accuracy by almost two points, which is more than the gap between the three classifiers and about as much as any tuning gain.
- **Test accuracy on the same kind of data overstates what was learned.** 88% in-domain became 65–70% on an unseen task type and chance on math. The classifiers had learned how each model formats each kind of answer more than a general signature.
- **Small measurement choices change conclusions.** A first count of LaTeX use matched dollar prices in the math answers and overstated it for two models; the rule had to be corrected.
- **Practical points.** Generation settings must be identical and recorded, the token limit needs a trial before the full run, and everything that produces a number should be a script that can be run again.

7. Limitations

- Three models, each a 4-bit local build with one system prompt and one temperature. Fingerprints of the same models served elsewhere may differ.
- One answer per prompt, English only, four prompt sources. The two Dolly task labels are loose.
- One training run per configuration on the test split.
- Embeddings are learned from scratch; pretrained embeddings and larger models were not tried.
- The explanations offered for the math result in RQ3 are hypotheses that were not tested.

References

- Y. Kim. Convolutional Neural Networks for Sentence Classification. EMNLP, 2014.
- S. Hochreiter and J. Schmidhuber. Long Short-Term Memory. Neural Computation, 9(8), 1997.
- K. Cobbe et al. Training Verifiers to Solve Math Word Problems. arXiv:2110.14168, 2021. (GSM8K)
- Databricks. databricks-dolly-15k. https://huggingface.co/datasets/databricks/databricks-dolly-15k, 2023.
- Ollama model library: https://ollama.com/library (llama3.1, qwen2.5, mistral).

- Game engines: https://github.com/godotengine/godot, https://github.com/OGRECave/ogre, https://github.com/TorqueGameEngines/Torque3D, https://github.com/BobbyAnguelov/Esoterica.

Appendix: reproducing the results

Step	Command	Output
Prompts	`python src/collect_prompts.py`	`data/prompts/prompts.jsonl`
Answers	`python src/generate_responses.py`	`data/responses/`
Checks	`python src/check_responses.py`	`results/response_stats.json`
Cleaning and split	`python src/build_dataset.py`	`data/splits/`
Baseline	`python src/baseline.py`	`results/baseline.json`
Settings search	`python src/tune.py`	`results/tuning.jsonl`
RQ1, RQ2	`python src/train.py --model cnn` (also `lstm`; `--mode prompt`, `--mode both`)	`results/runs/`
RQ3	`python src/train.py --model cnn --held-out math` (each model and task); `python src/baseline.py --held-out math`	`results/runs/`
RQ4	`python src/rq4.py`	`results/rq4.json`
Table and figures	`python src/evaluate.py`, `python src/plot_results.py`	`results/figures/`